

PRESIDENCY UNIVERSITY

Department of Computer Science and Engineering

CSE322

MICROPROCESSORS BASED SYSTEM DESIGN LABORATORY

Course Plan

Course Code	CSE322
Course Type	Laboratory
Number of Classes	12
Laboratory Focus	8086 Assembly Programming and Microprocessor-Based System Design

Prepared and Conducted by

Afzal Hossan

Lecturer

Department of Computer Science and Engineering
Presidency University

Academic Year: 2026

Contents

1	Course Overview	2
1.1	Course Progression	2
2	Course Objectives	2
3	Course Learning Outcomes (CLOs)	3
4	Course Structure	3
5	Detailed Laboratory Class Plan	4
5.1	Class 1: Registers, Counters & Buses	4
5.2	Class 2: SAP-1 Architecture	5
5.3	Class 3: SAP-1 Memory & Programming	6
5.4	Class 4: 8086 Architecture & Memory Organization	7
5.5	Class 5: 8086 Instruction Set & Addressing Modes	8
5.6	Class 6: 8086 Assembly Programming Fundamentals	9
5.7	Class 7: Arithmetic, Logic & Bit Manipulation	10
5.8	Class 8: Branching, Loops & Decision Making	11
5.9	Class 9: Arrays, Searching & Sorting	12
5.10	Class 10: Stack, Procedures & String Processing	12
5.11	Class 11: Advanced Assembly & I/O Programming	13
5.12	Class 12: RAM, Peripheral Interfacing & Comprehensive Practical	14
6	Recommended Laboratory Report Format	16
7	Required Laboratory Equipment and Software	16
7.1	Basic Equipment	16
7.2	Suggested Microprocessor Components	17
8	Assembly Programming Troubleshooting	17
8.1	Common Problems	17
8.2	Recommended Debugging Procedure	18
9	Assessment Scheme	18
10	Suggested Viva Assessment Areas	18
10.1	Registers, Counters & Buses	18
10.2	SAP-1	19
10.3	8086 Architecture	19
10.4	Instruction Set & Addressing Modes	19
10.5	Assembly Programming	19
10.6	Arithmetic & Logic	20
10.7	Branching, Loops & Arrays	20
10.8	Stack, Procedures & Strings	20
10.9	I/O & Interfacing	20
11	Course Concept Map	21
	Conclusion	21

1 Course Overview

CSE322: Microprocessors Based System Design Laboratory provides hands-on experience with fundamental concepts of microprocessor-based system design. The laboratory is designed to complement theoretical knowledge by allowing students to understand processor organization, memory organization, instruction execution, assembly language programming, and peripheral interfacing.

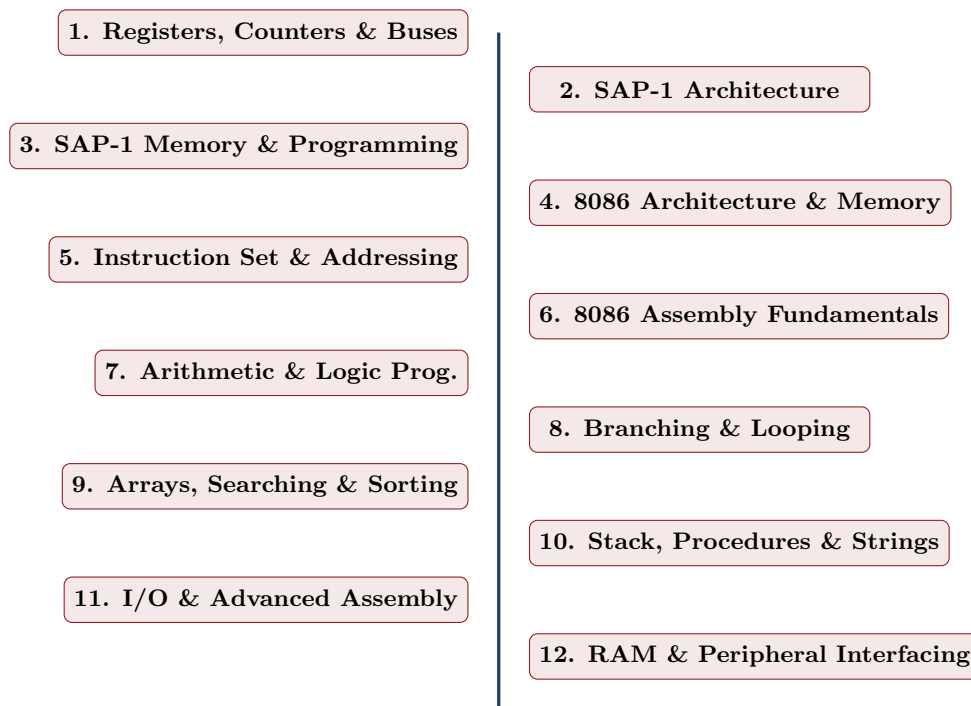
The course begins with fundamental processor components such as registers, counters, buses, and the ALU. Students are then introduced to the architecture and programming model of SAP-1 before progressing to the internal and external architecture of the 8086 microprocessor.

A major emphasis of the course is placed on **8086 instruction set, addressing modes, assembly language programming, debugging, and practical problem solving.**

The course concludes with basic memory and peripheral interfacing concepts and a comprehensive practical exercise integrating assembly programming with microprocessor-based system design.

1.1 Course Progression

The laboratory follows the sequence:



2 Course Objectives

The main objectives of this laboratory course are to:

1. Develop practical understanding of registers, counters, buses, and basic processor organization.
2. Understand the architecture and memory organization of SAP-1.
3. Trace the fetch-decode-execute cycle and basic SAP-1 instruction execution.
4. Understand the internal and external architecture of the 8086 microprocessor.
5. Develop understanding of 8086 memory segmentation and physical address calculation.

6. Understand the 8086 instruction set and different addressing modes.
7. Develop practical skills in 8086 assembly language programming.
8. Implement arithmetic, logical, bit manipulation, and control-flow operations using assembly language.
9. Develop programs involving arrays, searching, sorting, strings, procedures, and stack operations.
10. Develop basic keyboard and screen I/O programs using 8086 assembly language.
11. Introduce students to RAM organization, peripheral devices, and basic interfacing.
12. Integrate assembly programming and hardware concepts to solve microprocessor-based system design problems.

3 Course Learning Outcomes (CLOs)

After successful completion of the course, students will be able to:

CLO	Learning Outcome
CLO1	Identify and explain the roles of registers, counters, buses, ALU, and control units in a processor-based system.
CLO2	Analyze the architecture and memory organization of SAP-1 and trace basic instruction execution.
CLO3	Explain the internal and external architecture of the 8086 microprocessor and calculate physical memory addresses.
CLO4	Identify and apply appropriate 8086 instructions and addressing modes to solve programming problems.
CLO5	Write, assemble, execute, and debug basic 8086 assembly language programs.
CLO6	Develop assembly programs using arithmetic, logical, bit-manipulation, branching, and looping instructions.
CLO7	Develop assembly programs for arrays, searching, sorting, string processing, procedures, and stack operations.
CLO8	Implement basic input/output operations and understand RAM and peripheral interfacing concepts.
CLO9	Analyze program execution using registers, flags, memory, and processor state.
CLO10	Integrate assembly programming and microprocessor interfacing concepts to solve comprehensive practical problems.

4 Course Structure

Class	Topic	Major Practical Activity
1	Registers, Counters & Buses	Register transfer, bus operations, counters, and basic CPU tracing
2	SAP-1 Architecture	SAP-1 datapath analysis and instruction tracing
3	SAP-1 Memory & Programming	SAP-1 memory organization and basic program execution

Class	Topic	Major Practical Activity
4	8086 Architecture & Memory Organization	Architecture analysis and physical address calculation
5	8086 Instruction Set & Addressing Modes	Instruction identification and addressing-mode exercises
6	8086 Assembly Programming Fundamentals	Basic assembly programs and data manipulation
7	Arithmetic, Logic & Bit Manipulation	Arithmetic, logical, and bit-level programming
8	Branching, Loops & Decision Making	Conditional, iterative, and numerical programs
9	Arrays, Searching & Sorting	Array processing, searching, and sorting programs
10	Stack, Procedures & String Processing	Procedures, stack operations, and string programs
11	Advanced Assembly & I/O Programming	Keyboard/screen I/O and integrated assembly programs
12	RAM, Peripheral Interfacing & Comprehensive Practical	Memory/peripheral interfacing and integrated practical

5 Detailed Laboratory Class Plan

5.1 Class 1: Registers, Counters & Buses

Topics

- Registers and register organization
- Register transfer
- Register size
- General-purpose and special-purpose registers
- Common bus system
- Data bus
- Address bus
- Control bus
- Counters
- Program Counter
- ALU
- Control unit
- Basic CPU organization
- Instruction cycle

Practical Activities

Students will:

1. Identify different processor registers.
2. Demonstrate register-to-register data transfer.
3. Trace data movement through a common bus.
4. Identify data, address, and control buses.
5. Demonstrate counter operations.
6. Analyze the operation of a program counter.
7. Trace a simple instruction cycle.
8. Analyze the interaction among registers, ALU, control unit, and buses.

Experiment

Study and simulation of registers, counters, buses, and basic processor organization.

Expected Outcome

Students will be able to identify fundamental processor components and explain how data and control information move between them.

5.2 Class 2: SAP-1 Architecture

Topics

- SAP-1 architecture
- SAP-1 datapath
- 8-bit data bus
- Accumulator
- B register
- Program Counter
- Memory Address Register
- Instruction Register
- Output register
- ALU
- Control unit
- Memory organization
- Instruction format

Practical Activities

Students will:

1. Identify the major components of SAP-1.
2. Trace the SAP-1 data path.
3. Identify the function of each register.
4. Analyze the common 8-bit bus.
5. Trace data movement between registers.
6. Analyze the role of the ALU.
7. Trace basic SAP-1 instructions.

Experiment

Study and tracing of SAP-1 architecture and datapath operations.

Expected Outcome

Students will be able to identify SAP-1 components and trace data movement during instruction execution.

5.3 Class 3: SAP-1 Memory & Programming**Topics**

- SAP-1 memory organization
- Addressing memory locations
- Instruction encoding
- Opcode
- Operand
- Fetch-decode-execute cycle
- Microoperations
- Control signals
- LDA
- ADD
- SUB
- OUT
- HLT

Practical Activities

Students will:

1. Load values into SAP-1 memory.
2. Identify instruction and operand fields.

3. Trace the fetch cycle.
4. Trace the decode cycle.
5. Trace instruction execution.
6. Determine register contents after each microoperation.
7. Write and trace small SAP-1 programs.

Experiment

Programming and execution tracing of basic SAP-1 instructions.

Expected Outcome

Students will be able to trace SAP-1 programs and determine processor state during instruction execution.

5.4 Class 4: 8086 Architecture & Memory Organization

Topics

- Introduction to 8086
- CISC architecture
- Internal architecture
- External architecture
- Bus Interface Unit (BIU)
- Execution Unit (EU)
- General-purpose registers
- Segment registers
- Instruction Pointer
- FLAGS register
- Instruction queue
- Memory segmentation
- Physical address
- 20-bit addressing
- 1 MB memory space

Practical Activities

Students will:

1. Identify 8086 registers.
2. Identify the functions of segment registers.
3. Analyze the BIU and EU.
4. Trace instruction fetching.
5. Calculate physical addresses from segment and offset values.

6. Analyze segmented memory organization.
7. Trace basic instruction execution.

Experiment

Study of 8086 architecture and physical address calculation.

Expected Outcome

Students will be able to explain the major components of 8086 and calculate physical memory addresses.

5.5 Class 5: 8086 Instruction Set & Addressing Modes

Topics

- Data transfer instructions
- Arithmetic instructions
- Logical instructions
- Shift instructions
- Rotate instructions
- Control transfer instructions
- Stack instructions
- Immediate addressing
- Register addressing
- Direct addressing
- Register indirect addressing
- Based addressing
- Indexed addressing
- Based-indexed addressing

Practical Activities

Students will:

1. Identify different categories of instructions.
2. Determine the addressing mode used by an instruction.
3. Trace register contents after instruction execution.
4. Determine affected flags.
5. Analyze memory operands.
6. Convert simple problem statements into instruction sequences.

Experiment

Analysis and implementation of 8086 instructions using different addressing modes.

Expected Outcome

Students will be able to select appropriate instructions and addressing modes for assembly programming problems.

5.6 Class 6: 8086 Assembly Programming Fundamentals**Topics**

- Assembly language
- Assembly program structure
- Code segment
- Data segment
- Stack segment
- Directives
- Constants
- Variables
- Labels
- Comments
- Register operations
- Memory access
- MOV
- XCHG
- Basic arithmetic instructions
- Program assembly and execution
- Debugging

Practical Activities

Students will write and execute programs for:

1. Data transfer.
2. Register exchange.
3. Immediate data operations.
4. Memory-to-register operations.
5. Addition of two numbers.
6. Subtraction of two numbers.
7. 8-bit arithmetic.
8. 16-bit arithmetic.

Experiment

Writing, assembling, executing, and debugging basic 8086 assembly programs.

Expected Outcome

Students will be able to develop and execute basic 8086 assembly language programs.

5.7 Class 7: Arithmetic, Logic & Bit Manipulation**Topics**

- ADD
- ADC
- SUB
- SBB
- INC
- DEC
- MUL
- IMUL
- DIV
- IDIV
- AND
- OR
- XOR
- NOT
- TEST
- SHL/SAL
- SHR
- SAR
- ROL
- ROR
- Carry flag
- Zero flag
- Sign flag
- Overflow flag

Practical Activities

Students will implement programs for:

1. Multi-byte addition.
2. Multi-byte subtraction.
3. Multiplication.
4. Division.

5. Quotient and remainder calculation.
6. Even/odd detection.
7. Bit counting.
8. Setting, clearing, and toggling selected bits.

Experiment

Implementation of arithmetic, logical, and bit manipulation operations using 8086 assembly language.

Expected Outcome

Students will be able to implement numerical and bit-level operations using appropriate 8086 instructions.

5.8 Class 8: Branching, Loops & Decision Making

Topics

- FLAGS register
- CMP instruction
- Unconditional jump
- Conditional jumps
- LOOP instruction
- Nested loops
- Decision making
- Iterative programming

Practical Activities

Students will develop programs for:

1. Maximum of two numbers.
2. Maximum/minimum of an array.
3. Factorial.
4. Fibonacci sequence.
5. Prime number checking.
6. GCD.
7. LCM.
8. Multiplication table.
9. Digit manipulation.

Experiment

Implementation of decision-making and iterative algorithms using 8086 assembly language.

Expected Outcome

Students will be able to implement conditional and iterative algorithms using jumps and loops.

5.9 Class 9: Arrays, Searching & Sorting**Topics**

- Array representation
- Memory organization of arrays
- Indexed addressing
- SI and DI registers
- Sequential array processing
- Linear search
- Sorting
- Bubble sort
- Selection sort

Practical Activities

Students will develop programs for:

1. Array input and output.
2. Array summation.
3. Array average.
4. Maximum and minimum.
5. Counting positive/negative elements.
6. Counting even/odd elements.
7. Linear searching.
8. Bubble sorting.
9. Selection sorting.

Experiment

Array processing, searching, and sorting using 8086 assembly language.

Expected Outcome

Students will be able to manipulate arrays and implement basic searching and sorting algorithms in assembly language.

5.10 Class 10: Stack, Procedures & String Processing**Topics**

- Stack organization
- Stack Segment

- Stack Pointer
- PUSH
- POP
- CALL
- RET
- Procedures
- Parameter passing
- SI and DI
- String instructions
- REP
- String comparison
- String copying
- String reversal

Practical Activities

Students will implement:

1. PUSH and POP operations.
2. Stack-based data manipulation.
3. Procedures for arithmetic operations.
4. Procedures with parameters.
5. String copying.
6. String comparison.
7. String reversal.
8. Character searching.
9. Palindrome checking.

Experiment

Implementation of procedures, stack operations, and string-processing programs.

Expected Outcome

Students will be able to use the stack and procedures to develop modular assembly programs and process strings.

5.11 Class 11: Advanced Assembly & I/O Programming

Topics

- Keyboard input
- Screen output
- ASCII representation

- Character processing
- Decimal conversion
- Binary conversion
- Hexadecimal conversion
- Input validation
- Menu-driven programming
- Integration of procedures
- Arrays and I/O

Practical Activities

Students will develop:

1. Keyboard input programs.
2. Screen output programs.
3. ASCII processing programs.
4. Decimal input/output.
5. Decimal-to-binary conversion.
6. Decimal-to-hexadecimal conversion.
7. Menu-driven programs.
8. Simple calculator.
9. Integrated assembly applications.

Experiment

Implementation of I/O-based and integrated 8086 assembly language programs.

Expected Outcome

Students will be able to develop larger assembly programs combining procedures, control structures, data structures, and I/O operations.

5.12 Class 12: RAM, Peripheral Interfacing & Comprehensive Practical

Topics

- RAM organization
- Memory chips
- Address decoding
- Memory interfacing
- Peripheral devices
- I/O-mapped I/O
- Memory-mapped I/O
- Peripheral interfacing

- 8255 Programmable Peripheral Interface
- LED interfacing
- Switch interfacing
- Display interfacing

Practical Activities

Students should:

1. Analyze the given microprocessor-based problem.
2. Identify required inputs and outputs.
3. Determine memory/peripheral requirements.
4. Design the required interfacing scheme.
5. Develop the corresponding assembly program.
6. Implement the program.
7. Test the system.
8. Observe and record the results.
9. Troubleshoot hardware/software errors.
10. Explain the complete system during the viva.

Possible Comprehensive Problems

1. Develop an 8086 program for a numerical processing problem.
2. Implement an array searching or sorting application.
3. Develop a menu-driven assembly application.
4. Interface LEDs with 8086 through a peripheral interface.
5. Interface switches with 8086.
6. Implement a simple display-control application.
7. Develop a combined assembly and peripheral-interfacing system.

Experiment

Comprehensive microprocessor-based system design, assembly programming, interfacing, testing, and troubleshooting.

Expected Outcome

Students will be able to integrate 8086 assembly programming, memory organization, I/O, and peripheral interfacing to solve a comprehensive practical problem.

6 Recommended Laboratory Report Format

Each laboratory report should contain the following sections:

1. Experiment Number
2. Experiment Title
3. Date
4. Objective
5. Required Hardware/Software
6. Theoretical Background
7. Algorithm
8. Flowchart
9. Assembly Program
10. Program Explanation
11. Sample Input/Output
12. Experimental Results
13. Discussion
14. Error Analysis
15. Conclusion
16. Viva Questions

7 Required Laboratory Equipment and Software

7.1 Basic Equipment

- Personal computer
- 8086 microprocessor trainer kit, where available
- Microprocessor development board, where applicable
- Breadboard
- Regulated DC power supply
- Digital multimeter
- Oscilloscope, where applicable
- Connecting/jumper wires
- LEDs
- Resistors
- Push buttons/switches
- Seven-segment display
- Peripheral interface components

7.2 Suggested Microprocessor Components

Component	Purpose
8086	Microprocessor programming and system design
8255 PPI	Parallel peripheral interfacing
RAM	Memory organization and interfacing
ROM	Memory organization
LEDs	Output interfacing
Switches	Input interfacing
Seven-segment display	Display interfacing
Decoder / Address Decoder	Address selection
Clock Generator	Processor clock generation

Note: Component selection may be adjusted according to the available laboratory equipment.

8 Assembly Programming Troubleshooting

Students should be trained to systematically identify assembly-programming errors.

8.1 Common Problems

- Invalid instruction syntax
- Invalid operands
- Register-size mismatch
- Incorrect addressing mode
- Incorrect segment initialization
- Incorrect memory addressing
- Incorrect flag interpretation
- Incorrect conditional jump
- Infinite loops
- Stack overflow/underflow
- Incorrect procedure return
- Incorrect ASCII conversion
- Incorrect decimal/hexadecimal conversion
- Incorrect input handling
- Incorrect I/O operation

8.2 Recommended Debugging Procedure

Students should follow:

1. Identify the location of the error.
2. Check the instruction syntax.
3. Verify operand sizes.
4. Verify the addressing mode.
5. Check register contents.
6. Check the FLAGS register.
7. Check memory contents.
8. Trace the program instruction by instruction.
9. Verify loop and jump conditions.
10. Verify stack operations.
11. Test procedures independently.
12. Compare actual output with expected output.
13. Correct the error and execute the program again.

9 Assessment Scheme

A recommended assessment distribution is:

Assessment Component	Weight
Class Attendance	10%
Class Performance	10%
Lab Reports	20%
Field Assessment and Viva	20%
Final Practical Examination	40%
Total	100%

10 Suggested Viva Assessment Areas

The viva should evaluate both theoretical understanding and practical programming skills.

10.1 Registers, Counters & Buses

- What is a register?
- What is register transfer?
- What is a bus?
- What are data, address, and control buses?
- What is the function of a program counter?
- What is the role of the ALU?

10.2 SAP-1

- What is SAP-1?
- What are the major components of SAP-1?
- What is the function of the accumulator?
- What is the function of the instruction register?
- What is the fetch-decode-execute cycle?
- How is an SAP-1 instruction executed?

10.3 8086 Architecture

- What are the major components of 8086?
- What is the function of the BIU?
- What is the function of the EU?
- What are segment registers?
- What is memory segmentation?
- How is a physical address calculated?
- What is the purpose of the instruction queue?

10.4 Instruction Set & Addressing Modes

- What is an instruction set?
- What is an addressing mode?
- What is immediate addressing?
- What is register addressing?
- What is direct addressing?
- What is register-indirect addressing?
- What is indexed addressing?

10.5 Assembly Programming

- What is assembly language?
- What is an assembler?
- What is the difference between an instruction and a directive?
- What is the purpose of a label?
- Why are registers used?
- What is the purpose of the FLAGS register?

10.6 Arithmetic & Logic

- What is the difference between ADD and ADC?
- What is the difference between SUB and SBB?
- What is the difference between MUL and IMUL?
- What is the purpose of CMP?
- What is the difference between logical and arithmetic shifts?
- What are carry and overflow?

10.7 Branching, Loops & Arrays

- What is a conditional jump?
- What is the purpose of LOOP?
- How is an array represented in memory?
- What is the role of SI and DI?
- How can linear search be implemented?
- How can bubble sort be implemented?

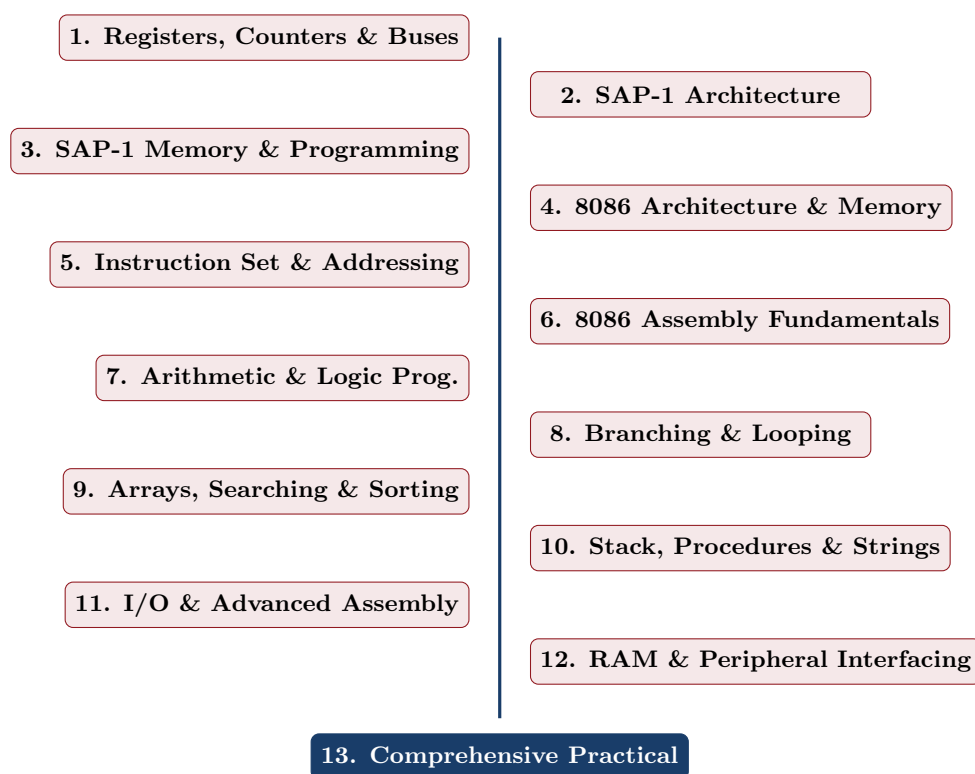
10.8 Stack, Procedures & Strings

- What is a stack?
- What are PUSH and POP?
- What is the purpose of CALL and RET?
- What is a procedure?
- How are parameters passed to procedures?
- What are string instructions?
- What is the purpose of REP?

10.9 I/O & Interfacing

- What is I/O?
- What is memory-mapped I/O?
- What is I/O-mapped I/O?
- What is a peripheral device?
- What is the purpose of 8255?
- How can an LED be interfaced?
- How can switches be used as input devices?

11 Course Concept Map



Conclusion

CSE322: Microprocessors Based System Design Laboratory is structured to provide a progressive practical foundation in microprocessor-based system design.

Students begin with fundamental processor components and the SAP-1 educational architecture, advance to the 8086 microprocessor, and progressively develop assembly programming skills. The later classes emphasize practical programming involving arithmetic, control flow, arrays, searching, sorting, stack operations, procedures, strings, and I/O.

The course concludes with RAM, peripheral interfacing, and an integrated comprehensive practical. The overall emphasis is on **programming, implementation, debugging, interfacing, and practical problem solving**.